Autonomous Network Configuration, Repair, and Traffic Optimization

Truc Nguyen and Nicholas D. Redmond
Department of Computer Science, University of Memphis, TN, USA
ttngyn12@memphis.edu and ndrdmond@memphis.edu

***Abstract*—Network outages and suboptimal performance are frequently a direct result of routing misconfigurations. To further complicate matters, there is a real-time challenge of fluctuating traffic demands that arises from a constantly growing, dynamic entity. Existing network verification tools can analyze *proposed configurations*, but superior operational recovery would require real-time monitoring, diagnosis, and adjustment. In this research project we propose a closed-loop system in which LLMs are fed the state of a network topology. The LLM proposes a change in configuration, which is evaluated by Batfish [1], and then measured after deployment. We seek to answer the question "Can an LLM autonomously identify, repair, and even improve a network's configuration and its traffic performance." We will compare static configurations, rule-based remediation, and LLM-assisted remediation across reproducible routing failures and congestion scenarios. Our expected contributions include a functional network testbed, a repeatable experimental framework, and an empirical assessment of repair accuracy, network performance, and recovery time.**

***Index Terms*—Configuration, Autonomous, Network Performance, LLM**

I. Introduction

Modern computer networks depend on complex interactions between routing protocols, device configurations, and dynamically changing traffic conditions. A single configuration error can interrupt connectivity, while inefficient routing decisions may degrade performance even when the network remains operational.

Traditional network management has relied on manual troubleshooting, rule-based automation, and specialized verification tools. However, recent advances in large language models have introduced the possibility of a system that can interpret network state, diagnose configuration problems, and generate corrective actions.

This project proposes a verification-guided, closed-loop network management system combining an LLM, the Batfish network verification framework [1], and a container-based network emulator [2]. The system will detect faults and performance degradation, propose configuration changes, verify those changes against predefined network requirements, and evaluate their effects using generated traffic.

The remainder of this proposal is organized as follows. Section II presents the motivation and background. Section III summarizes the project. Section IV gives the project details, including the architecture and environment, implementation issues and challenges, deliverables, and timeline. Section V concludes.

II. Motivation

Router misconfigurations remain a leading cause of network outages. In Facebook's 2021 outage, a single command issued during routine maintenance disconnected its entire backbone [3]. Routing protocols such as OSPF and BGP depend on consistent configuration across many devices, so error symptoms often appear far from the router that caused them. Network performance can also degrade because of congestion or inefficient path selection. These problems are particularly interesting because a successful corrective action requires understanding both local configurations and their network-wide consequences.

Existing solutions include network configuration analysis and verification systems such as Batfish, which can model forwarding behavior as well as identify violations of intended connectivity requirements but cannot repair them [1]. Recent research has studied using LLMs to generate network configurations, including NetConfEval [4]. More recent efforts, including Cornetto [5] and NetConfArena [6], demonstrate growing interest in evaluating LLM-based configuration repair and agentic network management. These studies also bring attention to the need to validate repairs rather than rely entirely on generated commands.

An opportunity for additional research is integrating LLM-driven decision-making, pre-deployment verification, and post-deployment performance feedback into a single experimental framework. The proposed system

will investigate whether its actions restore connectivity, improve traffic performance, avoid regressions, and complete within measurable recovery times.

III. Project Summary

This project will build and evaluate a closed-loop system in which an LLM agent diagnoses and repairs routing misconfigurations in an emulated network, deploying a change only after it passes formal verification and keeping it only if measured performance does not degrade. We will inject known routing, security, and performance faults into healthy networks and compare the full loop against single-shot LLM prompting and an agent without the safety gate, across a commercial and an open-source model. The evaluation will measure how accurately the agent locates each fault, how often it restores intended behavior, whether its changes introduce new connectivity or security failures, whether it improves link utilization and latency, how quickly it restores service, and how much the verification and measurement steps contribute relative to their cost. The project will produce an open-source framework, a set of fault scenarios with known solutions, and an evidence-based answer to whether verified LLM agents can safely repair and tune routing networks.

IV. Project Details

A. Architecture and Environment

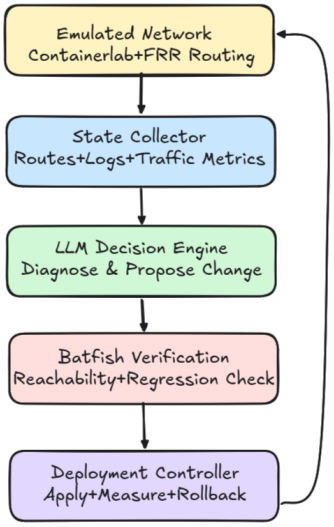


Fig. 1. Proposed architecture of closed-loop system.

The system is a closed loop of five components. A state collector gathers routing state, end-to-end throughput, latency, and loss from the emulated network, and reports anomalies. An LLM decision engine diagnoses the fault and proposes a configuration patch. Batfish checks each patch against invariants mined from the healthy network (required reachability, isolation between security zones, and required paths) and against the current configuration for regressions; rejected patches are returned to the LLM with the violations. A deployment controller applies approved patches, keeps a change only if the problem is resolved and performance has not degraded, and otherwise restores the last known-good configuration. It also records recovery time from fault injection to restored service. In congestion scenarios, the controller keeps a change only if it reduces link utilization or latency. We will create congestion scenarios, such as degrading a link with Linux traffic control or shifting traffic between zones, in which no configuration is wrong but traffic is routed inefficiently; the agent can respond by adjusting OSPF link costs to move traffic onto less loaded paths.

The emulated network runs FRRouting [7] routers in Containerlab [2], using OSPF for routing and interface ACLs to separate security zones. Our development network uses a four router topology; experiments will use the 11 router Abilene topology from the Internet Topology Zoo [8].

We will use an API to connect to a commercial model and use an open-source model. Batfish is accessed through *pybatfish*. All components are written in Python and run on a Linux workstation or virtual machine with sufficient resources to support the emulated routers, traffic generators, and verification service.

B. Implementation Issues and Challenges

A considerable amount of time is anticipated solving the interfacing between various software programs and LLM API's. The proposed architecture is home-baked and therefore only fits together in theory. We anticipate these challenges and have considered alternate software packages in the event of misalignment.

Containerlab will provide the execution environment, Batfish will evaluate supported configuration properties, and the LLM will generate bounded changes. A Python controller will connect these tools through a staged process: observe network state, diagnose the problem, propose a change, verify the candidate, deploy the change, measure the outcome, and roll back unsuccessful modifications. However, because Batfish analyzes modeled network behavior rather than actual traffic performance, runtime measurements will be necessary to validate any optimization decisions.

Our project is highly relevant in today's fast moving tech space and growing number of connected devices. We intend to realize the feasibility of a real-time decision

making system. One that is not only cognitive but can address long view tradeoffs in configuration management as well as anticipate traffic fluctuations based off its historical data. This idea of an intelligent network is very interesting and deserves the proper preliminary study.

C. Deliverables

The project will produce an emulated network testbed, a closed-loop framework that connects an LLM to the network through a verification step, and an experimental evaluation of how well the system repairs injected faults.

In the first phase, we will build the testbed, a small set of reproducible faults and congestion scenarios, and the Batfish verification step. The testbed is a small emulated network of virtual routers and hosts, built with Containerlab [2] and FRRouting [7], where faults and congestion can be introduced and the network reset to a healthy state between experiments.

In the second phase, we will connect the LLM, add deployment and rollback, and compare the full system against simpler baselines. For faults, we will measure repair success and safety; for congestion scenarios, we will measure whether the system improves performance by adjusting routing. The closed-loop framework collects the network's state, asks the LLM to diagnose the problem and propose a configuration change, checks that change with Batfish before applying it, and rolls it back if the network gets worse.

The final phase covers analysis, live demonstration, and the final report.

If time permits, we will extend the evaluation to additional decision models such as Jev [9], larger topologies, and more complex fault scenarios.

D. Proposed Division of Work

Nicholas Redmond

- *Network side:* building the emulated network, including the routers, hosts, links, and traffic generation, and the scripts that introduce faults and congestion.
- *Agent side:* building the LLM decision engine, including the prompts for both repair and routing optimization, the model integration, and the format of the diagnosis and proposed changes.

Truc Nguyen

- *Network side:* building the state collector that observes the network and measures performance, and the Batfish verification step that checks proposed changes.
- *Agent side:* building the deployment controller that applies, measures, and rolls back changes, including the rule for keeping a change only if performance improves, and the baseline versions of the system used for comparison.

Shared. Both members will build the first end-to-end version of the loop together, so that each understands the full system before working on separate parts. We will review each other's code weekly and share responsibility for the experiments, analysis, reports, and presentation, with each member presenting the components they built.

E. Timeline

Estimated timeline of deliverables and important dates:

- **Weeks 1–2:** Containerlab topology, initial traffic, and LLM interface.
- **Weeks 3–5:** Batfish checks, state collector, and fault scenarios.
- **Weeks 6–8:** Traffic impairment and deployment controller.
- **Final week:** Networking results and analysis

V. Conclusion

Recent studies show that LLMs can sometimes find and fix network configuration problems, but their solutions are often incomplete and may introduce new issues. Most research also reviews these fixes offline, without looking at their impact on real network traffic and performance. For this project, we will create a closed-loop system where an LLM agent interacts with a live, emulated network. The agent will watch the network, suggest configuration changes, and use Batfish to check each proposed fix for safety issues before making any changes. After a repair is applied, the system will measure how the network behaves to see if the change helped. We plan to test the system by adding network faults, using different LLM models, and trying out various setups with some parts left out. We will check how well the system diagnoses problems, how often repairs work, whether new issues or security risks appear, and how changes affect link use and latency.

VI. Acknowledgments

This proposal was written, in part, with assistance from GPT 6 in regards to grammar and style.

References

[1] A. Fogel, S. Fung, L. Pedrosa, M. Walraed-Sullivan, R. Govindan, R. Mahajan, and T. Millstein, "A general approach to network configuration analysis," in *Proc. 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, Oakland, CA, USA, May 2015, pp. 469–483.

[2] SRL Labs, “Containerlab: Container-based networking labs,” https://github.com/srl-labs/containerlab, 2026, accessed: Oct. 10, 2026.
[3] S. Janardhan, “More details about the October 4 outage,” Engineering at Meta, Oct. 2021, accessed: Oct. 10, 2026. [Online]. Available: https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/
[4] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, “NetConfEval: Can LLMs facilitate network configuration?” *Proceedings of the ACM on Networking*, vol. 2, no. CoNEXT2, Jun. 2024.
[5] I. Protogeros, R. Asadli, B. Hoffman, and L. Vanbever, “Benchmarking LLM-driven network configuration repair,” arXiv preprint arXiv:2604.22513, 2026.
[6] C. Liu, X. Xie, X. Chen, and Y. Cui, “NetConfArena: An executable benchmark for LLM agents in closed-loop network configuration,” arXiv preprint arXiv:2608.23179, Aug. 2026.
[7] FRRouting Project, “FRRouting: The frrouting protocol suite,” https://github.com/FRRouting/frr, 2026, accessed: Oct. 10, 2026.
[8] S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, and M. Roughan, “The internet topology zoo,” *IEEE Journal on Selected Areas in Communications*, vol. 29, no. 9, pp. 1765–1775, Oct. 2011.
[9] T. Deußer, L. Sparrenberg, and R. Sifa, “Evaluating and benchmarking the system one model jev,” 2026. [Online]. Available: https://arxiv.org/abs/2609.37647